

WEEK 1

1

PROBLEM STATEMENT:

This Java 8 challenge tests your knowledge of Lambda expressions!

Write the following methods that return a lambda expression performing a specified action:

1 PerformOperation isOdd(): The lambda expression must return true if a number is odd or false if it is even.

2 PerformOperation isPrime(): The lambda expression must return true if a number is prime or false if it is composite.

3 PerformOperation isPalindrome(): The lambda expression must return true if a number is a palindrome or false if it is not.

Input Format

Input is handled for you by the locked stub code in your editor.

Output Format

The locked stub code in your editor will print T lines of output.

Sample Input

The first line contains an integer, T (the number of test cases).

The T subsequent lines each describe a test case in the form of space-separated integers:

The first integer specifies the condition to check for (for Odd/Even, for Prime, or for Palindrome). The second integer denotes the number to be checked.

5

1 4

2 5

3 898

1 3

2 12

Sample Output

EVEN

PRIME

PALINDROME

ODD

COMPOSITE

CODE:

```
import java.io.*;

import java.util.*;

interface PerformOperation {

    boolean check(int a);

}

class MyMath {

    public static boolean checker(PerformOperation p, int num) {

        return p.check(num);

    }

    public PerformOperation isOdd() {

        return (int a) -> a % 2 != 0;

    }

    public PerformOperation isPrime() {

        return (int a) -> {

            if (a < 2) {

                return false;

            }

            for (int i = 2; i * i <= a; i++) {

                if (a % i == 0) {

                    return false;

                }

            }

            return true;

        };

    }

    public PerformOperation isPalindrome() {

        return (int a) -> {

            int original = a;

            int reverse = 0;
```

```

        while (a > 0) {
            int digit = a % 10;

            reverse = reverse * 10 + digit;

            a = a / 10;
        }

        return original == reverse;
    };
}

}

public class Solution {

    public static void main(String[] args) throws IOException {
        MyMath ob = new MyMath();

        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        int T = Integer.parseInt(br.readLine());

        PerformOperation op;

        boolean ret = false;

        String ans = null;

        while (T--> 0) {
            String s = br.readLine().trim();

            StringTokenizer st = new StringTokenizer(s);

            int ch = Integer.parseInt(st.nextToken());

            int num = Integer.parseInt(st.nextToken());

            if (ch == 1) {
                op = ob.isOdd();

                ret = ob.checker(op, num);

                ans = (ret) ? "ODD" : "EVEN";
            } else if (ch == 2) {
                op = ob.isPrime();

                ret = ob.checker(op, num);

                ans = (ret) ? "PRIME" : "COMPOSITE";
            }
        }
    }
}

```

```

    } else if (ch == 3) {
        op = ob.isPalindrome();
        ret = ob.checker(op, num);
        ans = (ret) ? "PALINDROME" : "NOT PALINDROME";

    }
    System.out.println(ans);
}
}
}
}

```

INPUT:

```

5
1 4
2 5
3 898
1 3
2 12

```

OUTPUT:

```

EVEN
PRIME
PALINDROME
ODD
COMPOSITE

```

2

PROBLEM STATEMENT:

You are given a list of student information: ID, FirstName, and CGPA. Your task is to rearrange them according to their CGPA in decreasing order. If two student have the same CGPA, then arrange them according to their first name in alphabetical order. If those two students also have the same first name, then order them according to their ID. No two students have the same ID.

Hint: You can use comparators to sort a list of objects. See the oracle docs to learn about comparators.

Input Format

The first line of input contains an integer , representing the total number of students. The next lines contains a list of student information in the following structure:

ID Name CGPA

Constraints

$2 \leq N \leq 1000$

$0 \leq ID \leq 100000$

$5 \leq |Name| \leq 30$

$0 \leq CGPA \leq 4.00$

The name contains only lowercase English letters. The contains only integer numbers without leading zeros. The CGPA will contain, at most, 2 digits after the decimal point.

Output Format

After rearranging the students according to the above rules, print the first name of each student on a separate line.

Sample Input

5

33 Rumpa 3.68

85 Ashis 3.85

56 Samiha 3.75

19 Samara 3.75

22 Fahim 3.76

Sample Output

Ashis

Fahim

Samara

Samiha

Rumpa

CODE:

```
import java.util.*;
```

```
class Student{
```

```
    private int id;
```

```
    private String fname;
```

```

private double cgpa;

public Student(int id, String fname, double cgpa) {
    super();
    this.id = id;
    this.fname = fname;
    this.cgpa = cgpa;
}

public int getId() {
    return id;
}

public String getFname() {
    return fname;
}

public double getCgpa() {
    return cgpa;
}
}

public class Solution
{
    public static void main(String[] args){
        Scanner in = new Scanner(System.in);

        int testCases = Integer.parseInt(in.nextLine())

        List<Student> studentList = new ArrayList<Student>();
        while(testCases>0){
            int id = in.nextInt();

            String fname = in.next();

            double cgpa = in.nextDouble();

            Student st = new Student(id, fname, cgpa);

            studentList.add(st);

            testCases--;
        }
    }
}

```

```

Collections.sort(studentList, new Comparator<Student>() {
public int compare(Student a, Student b) {
    if (a.getCgpa() != b.getCgpa())
        return Double.compare(b.getCgpa(), a.getCgpa());
    if (!a.getFname().equals(b.getFname()))
        return a.getFname().compareTo(b.getFname());
    return Integer.compare(a.getId(), b.getId());
}
});

for(Student st: studentList){
    System.out.println(st.getFname());
}
}

```

INPUT:

```

5
33 Rumpa 3.68
85 Ashis 3.85
56 Samiha 3.75
19 Samara 3.75
22 Fahim 3.76

```

OUTPUT:

```

    Ashis
    Fahim
    Samara
    Rumpa

```

PROBLEM STATEMENT:

Given an integer array nums, move all the even integers at the beginning of the array followed by all the odd integers.

Return any array that satisfies this condition.

Example 1:

Input: nums = [3,1,2,4]

Output: [2,4,3,1]

Explanation: The outputs [4,2,3,1], [2,4,1,3], and [4,2,1,3] would also be accepted.

Example 2:

Input: nums = [0]

Output: [0]

Constraints:

- $1 \leq \text{nums.length} \leq 5000$
- $0 \leq \text{nums}[i] \leq 5000$

INPUT:

nums =[3,1,2,4]

OUTPUT:

[2,4,3,1]

CODE:

```
import java.util.*;

class Player {
    String name;
    int score;

    Player(String name, int score) {
        this.name = name;
        this.score = score;
    }
}

class Checker implements Comparator<Player> {
    @Override
    public int compare(Player a, Player b) {
        if (a.score != b.score) {
            return b.score - a.score;
        }
    }
}
```

```

        return a.name.compareTo(b.name);
    }
}

public class Solution {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int n = scan.nextInt();
        Player[] player = new Player[n];
        Checker checker = new Checker();
        for(int i = 0; i < n; i++){
            player[i] = new Player(scan.next(), scan.nextInt());
        }
        scan.close();
        Arrays.sort(player, checker);
        for(int i = 0; i < player.length; i++){
            System.out.printf("%s %s\n", player[i].name, player[i].score);
        }
    }
}

```

INPUT:

5

amy 100

david 100

heraldo 50

aakansha 75

aleksa 150

OUTPUT:

aleksa 150

amy 100

david 100

aakansha 75

3

PROBLEM STATEMENT:

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array. The Player class is provided in the editor below. It has two fields:

name: a string.

score: an integer.

Given an array of Player objects, write a comparator that sorts them in order of decreasing score. If or more players have the same score, sort those players alphabetically ascending by name. To do this, you must create a Checker class that implements the Comparator interface, then write an `int compare(Player a, Player b)` method implementing the `Comparator.compare(T o1, T o2)` method. In short, when sorting in ascending order, a comparator function returns -1 if $a < b$, 0 if $a = b$, and 1 if $a > b$.

Declare a Checker class that implements the comparator method as described. It should sort first descending by score, then ascending by name. The code stub reads the input, creates a list of Player objects, uses your method to sort the data, and prints it out properly.

Example

`n=3 data=[[Smith,20],[Jones,15],[Jones,20]]`

Sort the list as `data sorted = [[Jones,20],[Smith,20],[Jones,15]]` . Sort first descending by score, then ascending by name.

Input Format

The first line contains an integer, `n` , the number of players.

Each of the next `n` lines contains a player's name and score , a string and an integer.

Constraints

$0 \leq \text{score} \leq 1000$

Two or more players can have the same name.

Player names consist of lowercase English alphabetic letters.

Output Format

You are not responsible for printing any output to stdout. Locked stub code in Solution will instantiate a Checker object, use it to sort the Player array, and print each sorted element.

Sample Input

5

amy 100

david 100

heraldo 50

aakansha 75

aleksa 150

Sample Output

aleksa 150

amy 100

david 100

aakansha 75

heraldo 50

Explanation

The players are first sorted descending by score, then ascending by name.

CODE:

```
import java.util.*;

class Player {
    String name;
    int score;

    Player(String name, int score) {
        this.name = name;
        this.score = score;
    }
}

class Checker implements Comparator<Player> {
    @Override
    public int compare(Player a, Player b) {
        if (a.score != b.score) {
            return b.score - a.score;
        }
        return a.name.compareTo(b.name);
    }
}

public class Solution {
```

```

public static void main(String[] args) {
    Scanner scan = new Scanner(System.in);
    int n = scan.nextInt();
    Player[] player = new Player[n];
    Checker checker = new Checker();
    for(int i = 0; i < n; i++){
        player[i] = new Player(scan.next(), scan.nextInt());
    }
    scan.close();
    Arrays.sort(player, checker);
    for(int i = 0; i < player.length; i++){
        System.out.printf("%s %s\n", player[i].name, player[i].score);
    }
}
}

```

INPUT:

```

5
amy 100
david 100
heraldo 50
aakansha 75
aleksa 150

```

OUTPUT:

```

aleksa 150
amy 100
david 100
aakansha 75
heraldo 50

```

PROBLEM STATEMENT:

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array.

The Player class is provided for you in your editor. It has fields: 2 a name String and a score integer.

Given an array of `Player` objects, write a comparator that sorts them in order of decreasing score; if 2 or more players have the same score, sort those players alphabetically by name. To do this, you must create a `Checker` class that implements the `Comparator` interface, then write an `int compare(Player a, Player b)` method implementing the `Comparator.compare(T o1, T o2)` method.

Input Format

Input from stdin is handled by the locked stub code in the Solution class.

The first line contains an integer, n , denoting the number of players.

Each of the n subsequent lines contains a player's name and score, respectively.

Constraints

0<=score<=1000	2
----------------	---

players can have the same name.

Player names consist of lowercase English letters.

Output Format

You are not responsible for printing any output to stdout. The locked stub code in Solution will create a Checker object, use it to sort the Player array, and print each sorted element.

Sample Input

5

amy 100

david 100

heraldo 50

aakansha 75

aleksa 150

Sample Output

aleksa 150

amy 100

david 100

aakansha 75

heraldo 50

CODE:

```
import java.util.*;

class Checker implements Comparator<Player> {

    @Override

    public int compare(Player a, Player b) {

        if (a.score != b.score) {

            return b.score - a.score;

        }

        return a.name.compareTo(b.name);

    }

}

class Player{

    String name;

    int score;

    Player(String name, int score){

        this.name = name;

        this.score = score;

    }

}

class Solution {

    public static void main(String[] args) {

        Scanner scan = new Scanner(System.in);

        int n = scan.nextInt();

        Player[] player = new Player[n];

        Checker checker = new Checker();

        for(int i = 0; i < n; i++){

            player[i] = new Player(scan.next(), scan.nextInt());

        }

        scan.close();

        Arrays.sort(player, checker);

    }

}
```

```

    for(int i = 0; i < player.length; i++){
        System.out.printf("%s %s\n", player[i].name, player[i].score);
    }
}

```

INPUT:

```

5
amy 100
david 100
heraldo 50
aakansha 75
aleksa 150

```

OUTPUT:

```

aleksa 150
amy 100
david 100
aakansha 75
heraldo 50

```

5

PROBLEM STATEMENT:

You are given an $m \times n$ integer grid `accounts` where `accounts[i][j]` is the amount of money the i^{th} customer has in the j^{th} bank. Return *the **wealth** that the richest customer has*.

A customer's **wealth** is the amount of money they have in all their bank accounts. The richest customer is the customer that has the maximum **wealth**

Example 1:

Input: `accounts = [[1,2,3],[3,2,1]]`

Output: 6

Explanation:

1st customer has $\text{wealth} = 1 + 2 + 3 = 6$

2nd customer has $\text{wealth} = 3 + 2 + 1 = 6$

Both customers are considered the richest with a wealth of 6 each, so return 6.

Example 2:

Input: accounts = [[1,5],[7,3],[3,5]]

Output: 10

Explanation:

1st customer has wealth = 6

2nd customer has wealth = 10

3rd customer has wealth = 8

The 2nd customer is the richest with a wealth of 10.

Example 3:

Input: accounts = [[2,8,7],[7,1,3],[1,9,5]]

Output: 17

Constraints:

- m == accounts.length
- n == accounts[i].length
- 1 <= m, n <= 50
- 1 <= accounts[i][j] <= 100

CODE:

```
class Solution {  
    public int maximumWealth(int[][] accounts) {  
        int maxWealth = 0;  
        for (int i = 0; i < accounts.length; i++) {  
            int wealth = 0;  
            for (int j = 0; j < accounts[i].length; j++) {  
                wealth += accounts[i][j];  
            }  
            maxWealth = Math.max(maxWealth, wealth);  
        }  
        return maxWealth;  
    }  
}
```

INPUT:

accounts = [[1,2,3],[3,2,1]]

OUTPUT:

6

WEEK 2

6

PROBLEM STATEMENT:-

Given a **zero-based permutation** **nums** (**0-indexed**), build an array **ans** of the **same length** where **ans[i] = nums[nums[i]]** for each $0 \leq i < \text{nums.length}$ and return it.

A **zero-based permutation** **nums** is an array of **distinct** integers from 0 to **nums.length - 1** (**inclusive**).

Example 1:

Input: **nums** = [0,2,1,5,3,4]

Output: [0,1,2,4,5,3]

Explanation: The array **ans** is built as follows:

```
ans = [nums[nums[0]], nums[nums[1]], nums[nums[2]], nums[nums[3]], nums[nums[4]],  
nums[nums[5]]]
```

```
= [nums[0], nums[2], nums[1], nums[5], nums[3], nums[4]]
```

```
= [0,1,2,4,5,3]
```

Example 2:

Input: **nums** = [5,0,1,2,3,4]

Output: [4,5,0,1,2,3]

Explanation: The array **ans** is built as follows:

```
ans = [nums[nums[0]], nums[nums[1]], nums[nums[2]], nums[nums[3]], nums[nums[4]],  
nums[nums[5]]]
```

```
= [nums[5], nums[0], nums[1], nums[2], nums[3], nums[4]]
```

```
= [4,5,0,1,2,3]
```

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $0 \leq \text{nums}[i] < \text{nums.length}$
- The elements in **nums** are **distinct**.

CODE:-

```

class Solution {
    public int[] buildArray(int[] nums) {

        return IntStream.range(0, nums.length)
            .map(i -> nums[nums[i]])
            .toArray();
    }
}

```

INPUT:

nums = [5,0,1,2,3,4]

OUTPUT:

[0,1,2,4,5,3]

PROBLEM STATEMENT:

Given the array nums consisting of $2n$ elements in the form $[x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n]$.

Return the array in the form $[x_1, y_1, x_2, y_2, \dots, x_n, y_n]$.

Example 1:

Input: nums = [2,5,1,3,4,7], n = 3

Output: [2,3,5,4,1,7]

Explanation: Since $x_1=2$, $x_2=5$, $x_3=1$, $y_1=3$, $y_2=4$, $y_3=7$ then the answer is [2,3,5,4,1,7].

Example 2:

Input: nums = [1,2,3,4,4,3,2,1], n = 4

Output: [1,4,2,3,3,2,4,1]

Example 3:

Input: nums = [1,1,2,2], n = 2

Output: [1,2,1,2]

Constraints:

- $1 \leq n \leq 500$

- `nums.length == 2n`
- `1 <= nums[i] <= 10^3`

CODE:

```
class Solution {
    public int[] shuffle(int[] nums, int n) {
        return IntStream.range(0, 2 * n)
            .map(i -> (i % 2 == 0) ? nums[i / 2] : nums[n + i / 2])
            .toArray();
    }
}
```

INPUT:

```
nums =
[2,5,1,3,4,7]
n = 3
```

OUTPUT:

```
[2,3,5,4,1,7]
```

7

PROBLEM STATEMENT:

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements should be kept the **same**.

Consider the number of *unique elements* in `nums` to be `k`. After removing duplicates, return the number of unique elements `k`.

The first `k` elements of `nums` should contain the unique numbers in **sorted order**. The remaining elements beyond index `k - 1` can be ignored.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length
int k = removeDuplicates(nums); // Calls your implementation
```

```
assert k == expectedNums.length;

for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

Input: nums = [1,1,2]

Output: 2, nums = [1,2,_]

Explanation: Your function should return k = 2, with the first two elements of nums being 1 and 2 respectively.

It does not matter what you leave beyond the returned k (hence they are underscores).

Example 2:

Input: nums = [0,0,1,1,1,2,2,3,3,4]

Output: 5, nums = [0,1,2,3,4,_,_,_,_,_]

Explanation: Your function should return k = 5, with the first five elements of nums being 0, 1, 2, 3, and 4 respectively.

It does not matter what you leave beyond the returned k (hence they are underscores).

Constraints:

- $1 \leq \text{nums.length} \leq 3 \times 10^4$
- $-100 \leq \text{nums}[i] \leq 100$
- nums is sorted in **non-decreasing** order.

CODE:

```
class Solution {
    public int removeDuplicates(int[] nums) {
        int k=1;

        int[] unique = Arrays.stream(nums)
            .filter(i -> true)
            .distinct()
            .toArray();

        System.arraycopy(unique, 0, nums, 0, unique.length);
    }
}
```

```

        return unique.length;
    }
}

```

INPUT:

nums = [1,1,2]

OUTPUT:

[1,2]

8

PROBLEM STATEMENT:

There is a biker going on a road trip. The road trip consists of $n + 1$ points at various altitudes. The biker starts his trip on point 0 with altitude equal 0.

You are given an integer array `gain` of length n where `gain[i]` is the **net gain in altitude** between points i and $i + 1$ for all $(0 \leq i < n)$. Return *the **highest altitude** of a point*.

Example 1:

Input: `gain = [-5,1,5,0,-7]`

Output: 1

Explanation: The altitudes are `[0,-5,-4,1,1,-6]`. The highest is 1.

Example 2:

Input: `gain = [-4,-3,-2,-1,4,3,2]`

Output: 0

Explanation: The altitudes are `[0,-4,-7,-9,-10,-6,-3,-1]`. The highest is 0.

Constraints:

- $n == \text{gain.length}$
- $1 \leq n \leq 100$
- $-100 \leq \text{gain}[i] \leq 100$

CODE:

```

class Solution {
    public int largestAltitude(int[] gain) {int altitude = 0;
        int highest = 0;

```

```

    for (int i = 0; i < gain.length; i++) {
        altitude = altitude + gain[i];
        if (altitude > highest) {
            highest = altitude;
        }
    }
    return highest;
}

```

INPUT:

gain = [-5,1,5,0,-7]

OUTPUT:

1

9

PROBLEM STATEMENT:

Given an integer array `nums` and an integer `k`, return *the k most frequent elements*. You may return the answer in **any order**

Example 1:

Input: `nums = [1,1,1,2,2,3]`, `k = 2`

Output: `[1,2]`

Example 2:

Input: `nums = [1]`, `k = 1`

Output: `[1]`

Example 3:

Input: `nums = [1,2,1,2,1,2,3,1,3,2]`, `k = 2`

Output: `[1,2]`

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

- k is in the range [1, the number of unique elements in the array].
- It is **guaranteed** that the answer is **unique**.

Follow up: Your algorithm's time complexity must be better than $O(n \log n)$, where n is the array's size

CODE:

```
import java.util.*;

class Solution {

    public int[] topKFrequent(int[] nums, int k) {

        HashMap<Integer, Integer> map = new HashMap<>();

        for (int num : nums) {

            map.put(num, map.getDefault(num, 0) + 1);

        }

        List<Integer>[] buckets = new List[nums.length + 1];

        for (int num : map.keySet()) {

            int frequency = map.get(num);

            if (buckets[frequency] == null) {

                buckets[frequency] = new ArrayList<>();

            }

            buckets[frequency].add(num);

        }

        int[] result = new int[k];

        int index = 0;

        for (int frequency = nums.length; frequency >= 1; frequency--) {

            if (buckets[frequency] != null) {

                for (int num : buckets[frequency]) {

                    result[index++] = num;

                    if (index == k) {

                        return result;

                    }

                }

            }

        }

    }

}
```

```

    }
    return result;
}
}

```

INPUT:

nums = [1,1,1,2,2,3]

OUTPUT:

k =2

10

PROBLEM STATEMENT:

In computer science, a double-ended queue (dequeue, often abbreviated to deque, pronounced deck) is an abstract data type that generalizes a queue, for which elements can be added to or removed from either the front (head) or back (tail).

Deque interfaces can be implemented using various types of collections such as LinkedList or ArrayDeque classes. For example, deque can be declared as:

```
Deque deque = new LinkedList<>();
```

or

```
Deque deque = new ArrayDeque<>();
```

You can find more details about Deque [here](#).

In this problem, you are given N integers. You need to find the maximum number of unique integers among all the possible contiguous subarrays of size M .

Note: Time limit is 3 second for this problem.

Input Format

The first line of input contains two integers N and M : representing the total number of integers and the size of the subarray, respectively. The next line contains N space separated integers.

Constraints

$1 \leq N \leq 100000$

$1 \leq m \leq 100000$

$M \leq N$

The numbers in the array will range between $[0, 10000000]$.

Output Format

Print the maximum number of unique integers among all possible contiguous subarrays of size M .

Sample Input

6 3

5 3 5 2 3 2

Sample Output

3

Explanation

In the sample testcase, there are 4 subarrays of contiguous numbers.

s1=(5,3,5)- Has 2 unique numbers.

s2=(3,5,2) - Has 3 unique numbers.

s3=(5,2,3) - Has 3 unique numbers.

s4=(2,3,2)- Has 2 unique numbers.

In these subarrays, there are 2,3,3,2 unique numbers, respectively. The maximum amount of unique numbers among all possible contiguous subarrays is 3.

CODE:

```
import java.util.*;

public class test {

    public static void main(String[] args) {

        Scanner in = new Scanner(System.in);

        int n = in.nextInt();

        int m = in.nextInt();

        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {

            arr[i] = in.nextInt();

        }

        Deque<Integer> deque = new ArrayDeque<>();

        Map<Integer, Integer> freq = new HashMap<>();

        int maxUnique = 0;

        for (int i = 0; i < m; i++) {

            deque.addLast(arr[i]);

            freq.put(arr[i], freq.getOrDefault(arr[i], 0) + 1);
```

```

    }

    maxUnique = freq.size();
    for (int i = m; i < n; i++) {
        int removed = deque.removeFirst();
        freq.put(removed, freq.get(removed) - 1);
        if (freq.get(removed) == 0) {
            freq.remove(removed);
        }
        deque.addLast(arr[i]);
        freq.put(arr[i], freq.getOrDefault(arr[i], 0) + 1);
        maxUnique = Math.max(maxUnique, freq.size());
    }

    System.out.println(maxUnique);
    in.close();
}
}

```

INPUT:

```

6 3
5 3 5 2 3 2

```

OUTPUT:

```

3

```

PROBLEM STATEMENT:

In computer science, a set is an abstract data type that can store certain values, without any particular order, and no repeated values(Wikipedia). {1,2,3} is an example of a set, but {1,2,2} is not a set. Today you will learn how to use sets in java by solving this problem.

You are given pairs of strings. Two (a,b) pairs and (c,d) are identical if a=c and b=d. That also implies (a,b) is not same as (b,a) . After taking each pair as input, you need to print number of unique pairs you currently have.

Complete the code in the editor to solve this problem.

Input Format

In the first line, there will be an integer T denoting number of pairs. Each of the next T lines will contain two strings seperated by a single space.

Constraints:

$1 \leq T \leq 100000$

Length of each string is atmost 5 and will consist lower case letters only.

Output Format

Print T lines. In the i th line, print number of unique pairs you have after taking ith pair as input.

Sample Input

5

john tom

john mary

john tom

mary anna

mary anna

Sample Output

1

2

2

3

3

Explanation

After taking the first input, you have only one pair: (john,tom)

After taking the second input, you have two pairs: (john, tom) and (john, mary)

After taking the third input, you still have two unique pairs.

After taking the fourth input, you have three unique pairs: (john,tom), (john, mary) and (mary, anna)

After taking the fifth input, you still have three unique pairs.

CODE:

```
import java.io.*;
```

```
import java.util.*;
```

```
import java.text.*;
```

```
import java.math.*;
```

```
import java.util.regex.*;
```

```

public class Solution {

    public static void main(String[] args) {

        Scanner s = new Scanner(System.in);

        int t = s.nextInt();

        String [] pair_left = new String[t];

        String [] pair_right = new String[t];

        for (int i = 0; i < t; i++) {

            pair_left[i] = s.next();

            pair_right[i] = s.next();

        }

        HashSet<String> set = new HashSet<>();

        for (int i = 0; i < t; i++) {

            String pair = pair_left[i] + " " + pair_right[i];

            set.add(pair);

            System.out.println(set.size());

        }

        s.close();

    }

}

```

INPUT:

```

5
john tom
john mary
john tom
mary anna
mary anna

```

OUTPUT:

```

1
2
2

```

3

3

WEEK 3

11

PROBLEM STATEMENT:

The Calendar class is an abstract class that provides methods for converting between a specific instant in time and a set of calendar fields such as YEAR, MONTH, DAY_OF_MONTH, HOUR, and so on, and for manipulating the calendar fields, such as getting the date of the next week.

You are given a date. You just need to write the method, `getDay`, which returns the day on that date. To simplify your task, we have provided a portion of the code in the editor.

Example

`month=8`

`day=14`

`year=2017`

The method should return MONDAY as the day on that date.

AUGUST 2017						
SUN	MON	TUE	WED	THU	FRI	SAT
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	31		

Function Description

Complete the `findDay` function in the editor below.

`findDay` has the following parameters:

`int`: month

`int`: day

`int`: year

Returns

string: the day of the week in capital letters

Input Format

A single line of input containing the space separated month, day and year, respectively, in MM DD YYYY format.

Constraints

2000<year<3000

Sample Input

08 05 2015

Sample Output

WEDNESDAY

Explanation

The day on August 5th 2015 was WEDNESDAY.

CODE:

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    public static String findDay(int month, int day, int year) {

        Calendar cal = Calendar.getInstance();

        cal.set(year, month - 1, day);

        String[] days = {
```

```

        "SUNDAY",
        "MONDAY",
        "TUESDAY",
        "WEDNESDAY",
        "THURSDAY",
        "FRIDAY",
        "SATURDAY"
    };

    return days[cal.get(Calendar.DAY_OF_WEEK) - 1];
}
}

public class Solution {

    public static void main(String[] args) throws IOException {

        BufferedReader bufferedReader = new BufferedReader(new
        InputStreamReader(System.in));

        BufferedWriter bufferedWriter = new BufferedWriter(new
        FileWriter(System.getenv("OUTPUT_PATH")));

        String[] firstMultipleInput = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");

        int month = Integer.parseInt(firstMultipleInput[0]);

        int day = Integer.parseInt(firstMultipleInput[1]);

        int year = Integer.parseInt(firstMultipleInput[2]);

        String res = Result.findDay(month, day, year);

        bufferedWriter.write(res);

        bufferedWriter.newLine();

        bufferedReader.close();

        bufferedWriter.close();

    }
}

```

INPUT:

08 05 2015

OUTPUT:

WEDNESDAY

PROBLEM STATEMENT:

Write a program to count the number of days between two dates.

The two dates are given as strings, their format is YYYY-MM-DD as shown in the examples.

Example 1:

Input: date1 = "2019-06-29", date2 = "2019-06-30"

Output: 1

Example 2:

Input: date1 = "2020-01-15", date2 = "2019-12-31"

Output: 15

Constraints:

- The given dates are valid dates between the years 1971 and 2100

CODE:

```
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
class Solution {
    public int daysBetweenDates(String date1, String date2) {
        LocalDate d1 = LocalDate.parse(date1);
        LocalDate d2 = LocalDate.parse(date2);
        return (int) Math.abs(ChronoUnit.DAYS.between(d1, d2));
    }
}
```

INPUT:

date1 ="2019-06-29"

date2 ="2019-06-30"

OUTPUT:

1

PROBLEM STATEMENT:

Given a string date representing a [Gregorian calendar](#) date formatted as YYYY-MM-DD, return *the day number of the year*.

Example 1:

Input: date = "2019-01-09"

Output: 9

Explanation: Given date is the 9th day of the year in 2019.

Example 2:

Input: date = "2019-02-10"

Output: 41

Constraints:

- date.length == 10
- date[4] == date[7] == '-', and all other date[i]'s are digits
- date represents a calendar date between Jan 1st, 1900 and Dec 31st, 2019.

CODE:

```
import java.time.LocalDate;

class Solution {
    public int dayOfYear(String date) {
        LocalDate d = LocalDate.parse(date);
        return d.getDayOfYear();
    }
}
```

INPUT:

date="2019-01-09"

OUTPUT:

9

PROBLEM STATEMENT:

Given a date, return the corresponding day of the week for that date.

The input is given as three integers representing the day, month and year respectively.

Return the answer as one of the following values {"Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"}.

Note: January 1, 1971 was a Friday.

Example 1:

Input: day = 31, month = 8, year = 2019

Output: "Saturday"

Example 2:

Input: day = 18, month = 7, year = 1999

Output: "Sunday"

Example 3:

Input: day = 15, month = 8, year = 1993

Output: "Sunday"

Constraints:

- The given dates are valid dates between the years 1971 and 2100.

CODE:

```
import java.time.LocalDate;

class Solution {

    public String dayOfTheWeek(int day, int month, int year) {

        LocalDate date = LocalDate.of(year, month, day);

        return date.getDayOfWeek().toString().substring(0, 1)

            + date.getDayOfWeek().toString().substring(1).toLowerCase();

    }

}
```

INPUT:

day =31

month =8

year =2019

OUTPUT:

"Saturday"

PROBLEM STATEMENT:

In computer science, a priority queue is an abstract data type which is like a regular queue, but where additionally each element has a "priority" associated with it. In a priority queue, an element with high priority is served before an element with low priority. - [Wikipedia](#)

In this problem we will test your knowledge on [Java Priority Queue](#).

There are a number of students in a school who wait to be served. Two types of events, *ENTER* and *SERVED*, can take place which are described below.

- *ENTER*: A student with some priority enters the queue to be served.
- *SERVED*: The student with the highest priority is served (removed) from the queue.

A unique id is assigned to each student entering the queue. The queue serves the students based on the following criteria (priority criteria):

1. The student having the highest *Cumulative Grade Point Average* (CGPA) is served first.
2. Any students having the *same CGPA* will be served by name in ascending case-sensitive alphabetical order.
3. Any students having the *same CGPA and name* will be served in ascending order of the id.

Create the following two classes:

- The *Student* class should implement:
 - The constructor `Student(int id, String name, double cgpa)`.
 - The method `int getID()` to return the id of the student.
 - The method `String getName()` to return the name of the student.
 - The method `double getCGPA()` to return the CGPA of the student.
- The *Priorities* class should implement the method `List<Student> getStudents(List<String> events)` to process all the given events and return all the students yet to be served in the priority order.

Input Format

The first line contains an integer, `n`, describing the total number of events. Each of the subsequent lines will be of the following two forms:

- `ENTER name CGPA id`: The student to be inserted into the priority queue.
- `SERVED`: The highest priority student in the queue was served.

The locked stub code in the editor reads the input and tests the correctness of the *Student* and *Priorities* classes implementation.

Constraints

- $2 \leq n \leq 1000$
- $0 \leq \text{CGPA} \leq 4.00$
- $1 \leq \text{id} \leq 10^5$
- $2 \leq |\text{name}| \leq 30$

Output Format

The locked stub code prints the names of the students yet to be served in the priority order. If there are no such student, then the code prints EMPTY.

Sample Input 0

```
12
ENTER John 3.75 50
ENTER Mark 3.8 24
ENTER Shafaet 3.7 35
SERVED
SERVED
ENTER Samiha 3.85 36
SERVED
ENTER Ashley 3.9 42
ENTER Maria 3.6 46
ENTER Anik 3.95 49
ENTER Dan 3.95 50
SERVED
```

Sample Output 0

```
Dan
Ashley
Shafaet
Maria
```

Explanation 0

In this case, the number of events is 12. Let the name of the queue be Q .

- John is added to Q . So, it contains $(John, 3.75, 50)$.
- Mark is added to Q . So, it contains $(John, 3.75, 50)$ and $(Mark, 3.8, 24)$.
- Shafaet is added to Q . So, it contains $(John, 3.75, 50)$, $(Mark, 3.8, 24)$, and $(Shafaet, 3.7, 35)$.

- Mark is served as he has the highest CGPA. So, Q contains (*John*, 3.75, 50) and (*Shafaet*, 3.7, 35).
- John is served next as he has the highest CGPA. So, Q contains (*Shafaet*, 3.7, 35).
- Samiha is added to Q. So, it contains (*Shafaet*, 3.7, 35) and (*Samiha*, 3.85, 36).
- Samiha is served as she has the highest CGPA. So, Q contains (*Shafaet*, 3.7, 35).
- Now, four more students are added to Q. So, it contains (*Shafaet*, 3.7, 35), (*Ashley*, 3.9, 42), (*Maria*, 3.6, 46), (*Anik*, 3.95, 49), and (*Dan*, 3.95, 50).
- Anik is served because though both Anil and Dan have the highest CGPA but Anik comes first when sorted in alphabetic order. So, Q contains (*Dan*, 3.95, 50), (*Ashley*, 3.9, 42), (*Shafaet*, 3.7, 35), and (*Maria*, 3.6, 46).

As all events are completed, the name of each of the remaining students is printed on a new line.

CODE:

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;
class Student {
    private int id;
    private String name;
    private double cgpa;
    public Student(int id, String name, double cgpa) {
        this.id = id;
        this.name = name;
        this.cgpa = cgpa;
    }
    public int getID() {
        return id;
    }
    public String getName() {
        return name;
    }
    public double getCGPA() {
        return cgpa;
    }
}
```

```

    }
}

class Priorities {

    public List<Student> getStudents(List<String> events) {

        java.util.PriorityQueue<Student> pq =

            new java.util.PriorityQueue<Student>(

                (a, b) -> {

                    if (a.getCGPA() > b.getCGPA()) {

                        return -1;

                    }

                    if (a.getCGPA() < b.getCGPA()) {

                        return 1;

                    }

                    int nameCompare =

                        a.getName().compareTo(b.getName());

                    if (nameCompare != 0) {

                        return nameCompare;

                    }

                    return Integer.compare(a.getID(), b.getID());

                }

            );

        for (String event : events) {

            String[] parts = event.split(" ");

            if (parts[0].equals("ENTER")) {

                String name = parts[1];

                double cgpa = Double.parseDouble(parts[2]);

                int id = Integer.parseInt(parts[3]);

                pq.add(new Student(id, name, cgpa));

            } else if (parts[0].equals("SERVED")) {

                if (!pq.isEmpty()) {

                    pq.poll();

                }

            }

        }

    }

}

```

```

        }
    }
}
List<Student> result = new ArrayList<>();
while (!pq.isEmpty()) {
    result.add(pq.poll());
}
return result;
}
}

public class Solution {
    private final static Scanner scan = new Scanner(System.in);
    private final static Priorities priorities = new Priorities();
    public static void main(String[] args) {
        int totalEvents = Integer.parseInt(scan.nextLine());
        List<String> events = new ArrayList<>();
        while (totalEvents-- != 0) {
            String event = scan.nextLine();
            events.add(event);
        }
        List<Student> students = priorities.getStudents(events);
        if (students.isEmpty()) {
            System.out.println("EMPTY");
        } else {
            for (Student st: students) {
                System.out.println(st.getName());
            }
        }
    }
}

```

INPUT:

12
ENTER John 3.75 50
ENTER Mark 3.8 24
ENTER Shafaet 3.7 35
SERVED
SERVED
ENTER Samiha 3.85 36
SERVED
ENTER Ashley 3.9 42
ENTER Maria 3.6 46
ENTER Anik 3.95 49
ENTER Dan 3.95 50
SERVED

OUTPUT:

Dan
Ashley
Shafaet
Maria

PROBLEM STATEMENT:

Sometimes it's better to use dynamic size arrays. Java's ArrayList can provide you this feature. Try to solve this problem using ArrayList.

You are given n lines. In each line there are zero or more integers. You need to answer a few queries where you need to tell the number located in y th position x th of line.

Take your input from System.in.

Input Format :- The first line has an integer n . In each of the next n lines there will be an integer d denoting number of integers on that line and then there will be d space-separated integers. In the next line there will be an integer q denoting number of queries. Each query will consist of two integers x and y .

Constraints

$1 \leq n \leq 20000$

$0 \leq d \leq 50000$

$1 \leq q \leq 10000$

$1 \leq x \leq n$

Each number will fit in signed integer. Total number of integers in n lines will not cross 10^5 .

Output Format:- In each line, output the number located y th in position of x th line. If there is no such position, just print "ERROR!"

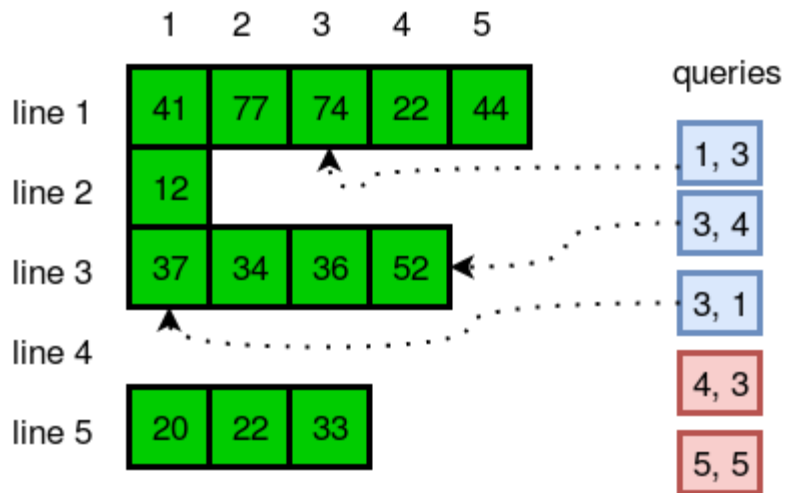
Sample Input

```
5
5 41 77 74 22 44
1 12
4 37 34 36 52
0
3 20 22 33
5
1 3
3 4
3 1
4 3
5 5
```

Sample Output

```
74
52
37
ERROR!
ERROR!
```

Explanation



The diagram below explains the queries:

CODE:

```
import java.io.*;
import java.util.*;

public class Solution {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        ArrayList<ArrayList<Integer>> lines =
            new ArrayList<>();

        for (int i = 0; i < n; i++) {
            int d = sc.nextInt();

            ArrayList<Integer> list = new ArrayList<>();

            for (int j = 0; j < d; j++) {
                list.add(sc.nextInt());
            }

            lines.add(list);
        }

        int q = sc.nextInt();

        for (int i = 0; i < q; i++) {
            int x = sc.nextInt();
            int y = sc.nextInt();
```

```

        if (x >= 1 && x <= n &&
            y >= 1 && y <= lines.get(x - 1).size()) {
            System.out.println(lines.get(x - 1).get(y - 1));
        } else {
            System.out.println("ERROR!");
        }
    }
}
sc.close();
}
}

```

INPUT:

```

5
5 41 77 74 22 44
1 12
4 37 34 36 52
0
3 20 22 33
5
1 3
3 4
3 1
4 3
5 5

```

OUTPUT:

```

74
52
37
ERROR!
ERROR!

```

PROBLEM STATEMENT:

Given a list of non-negative integers `nums`, arrange them such that they form the largest number and return it.

Since the result may be very large, so you need to return a string instead of an integer.

Example 1:

Input: `nums = [10,2]`

Output: "210"

Example 2:

Input: `nums = [3,30,34,5,9]`

Output: "9534330"

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 10^9$

CODE:

```
import java.util.*;

class Solution {
    public String largestNumber(int[] nums) {
        String[] arr = new String[nums.length];
        for (int i = 0; i < nums.length; i++) {
            arr[i] = String.valueOf(nums[i]);
        }
        Arrays.sort(arr, (a, b) -> (b + a).compareTo(a + b));
        if (arr[0].equals("0")) {
            return "0";
        }
        StringBuilder result = new StringBuilder();
        for (String s : arr) {
            result.append(s);
        }
        return result.toString();
    }
}
```

```
}  
}
```

INPUT:

nums =[10,2]

OUTPUT:

"210"

PROBLEM STATEMENT:

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array.

The Player class is provided for you in your editor. It has 2 fields: a name String and a score integer.

Given an array of n Player objects, write a comparator that sorts them in order of decreasing score; if 2 or more players have the same score, sort those players alphabetically by name. To do this, you must create a Checker class that implements the Comparator interface, then write an int compare(Player a, Player b) method implementing the Comparator.compare(T o1, T o2) method.

Input Format

Input from stdin is handled by the locked stub code in the Solution class.

The first line contains an integer, n , denoting the number of players.

Each of the n subsequent lines contains a player's name and score, respectively

Constraints

$0 \leq \text{score} \leq 1000$

2 players can have the same name.

Player names consist of lowercase English letters.

Output Format

You are not responsible for printing any output to stdout. The locked stub code in Solution will create a Checker object, use it to sort the Player array, and print each sorted element.

Sample Input

5

amy 100

david 100

heraldo 50

aakansha 75

aleksa 150

Sample Output

aleksa 150

amy 100

david 100

aakansha 75

heraldo 50

CODE:

```
import java.util.*;

class Checker implements Comparator<Player> {

    @Override

    public int compare(Player a, Player b) {

        if (a.score != b.score) {

            return b.score - a.score;

        }

        return a.name.compareTo(b.name);

    }

}

class Player{

    String name;

    int score;

    Player(String name, int score){

        this.name = name;

        this.score = score;

    }

}

class Solution {

    public static void main(String[] args) {

        Scanner scan = new Scanner(System.in);

        int n = scan.nextInt();
```

```

Player[] player = new Player[n];
Checker checker = new Checker();
for(int i = 0; i < n; i++){
    player[i] = new Player(scan.next(), scan.nextInt());
}
scan.close();
Arrays.sort(player, checker);
for(int i = 0; i < player.length; i++){
    System.out.printf("%s %s\n", player[i].name, player[i].score);
}
}
}

```

INPUT:

```

5
amy 100
david 100
heraldo 50
aakansha 75
aleksa 150

```

OUTPUT:

```

aleksa 150
amy 100
david 100
aakansha 75
heraldo 50

```

15

PROBLEM STATEMENT:

You are given an array of strings names, and an array heights that consists of **distinct** positive integers. Both arrays are of length n.

For each index i, names[i] and heights[i] denote the name and height of the ith person.

Return names sorted in **descending** order by the people's heights.

Example 1:

Input: names = ["Mary","John","Emma"], heights = [180,165,170]

Output: ["Mary","Emma","John"]

Explanation: Mary is the tallest, followed by Emma and John.

Example 2:

Input: names = ["Alice","Bob","Bob"], heights = [155,185,150]

Output: ["Bob","Alice","Bob"]

Explanation: The first Bob is the tallest, followed by Alice and the second Bob.

Constraints:

- $n == \text{names.length} == \text{heights.length}$
- $1 \leq n \leq 10^3$
- $1 \leq \text{names}[i].\text{length} \leq 20$
- $1 \leq \text{heights}[i] \leq 10^5$
- `names[i]` consists of lower and upper case English letters.
- All the values of heights are distinct.

CODE:

```
import java.util.*;
```

```
class Solution {  
    public String[] sortPeople(String[] names, int[] heights) {  
        Integer[] index = new Integer[names.length];  
        for (int i = 0; i < names.length; i++) {  
            index[i] = i;  
        }  
        Arrays.sort(index, (a, b) -> heights[b] - heights[a]);  
        String[] result = new String[names.length];  
        for (int i = 0; i < names.length; i++) {  
            result[i] = names[index[i]];  
        }  
        return result;  
    }  
}
```

```
}  
}
```

INPUT:

names=["Mary","John","Emma"]

heights=[180,165,170]

OUTPUT:

["Mary","Emma","John"]